



PEJU/AND/ADA/APP WEB IA

DESARROLLO DE APLICACIONES WEB CON IA

M.3011.002.083

Desarrollo Backend - Express

Juan Manuel Castillo Andrades



Cofinanciado por
la Unión Europea



Fondos Europeos



Sistema Nacional de
Garantía Juvenil



Express

A bright yellow square containing the letters 'JS' in a bold, black, sans-serif font.

JS



*“Es un **framework** para Node que **facilita** ... like a charm ... la creación de aplicaciones webs y API's”*





express Instalación

express - Creación de un nuevo proyecto

```
// creamos la carpeta de nuestro proyecto y accedemos a ella
> mkdir my_first_expressjs_app
> cd my_first_expressjs_app

// iniciamos el proyecto con yarn
> npm init

// instalamos el paquete de express
> npm install express
```

express - Hola mundo

```
import express from 'express'

const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('¡Hola mundo!')
})

app.listen(port, () => {
  console.log(`Servidor activo en http://localhost:${port}!`)
})
```



express

Características



express - Características

- Enfocado a **rutas**
- Uso intensivo de **middlewares**
- Permite el uso de **motores de templating**
- Gestión de **estáticos** y **errores**
- Flexibilidad en la integración con **bases de datos**



express - A qué no nos ayuda

- **Autenticación** de usuarios
- **Seguridad**
- **Despliegues** (subir nuestro proyecto a un servidor remoto)
- **Organización** del código



express
Enrutado



express - Sistema de enrutado

Es un sistema básico de cualquier framework orientado a servicios web ya que cualquier solicitud sobre la información (consultar, modificar, eliminar...) parte de una petición a una url (dónde está el recurso).

El sistema de enrutado, se encarga de ejecutar la parte del código que nosotros especifiquemos cuando se llama a una ruta.

Ruta
<code>http://localhost</code>
<code>http://localhost/contacto</code>
<code>http://localhost/quienes-somos</code>
<code>http://localhost/proyectos</code>
<code>http://localhost/presupuesto</code>



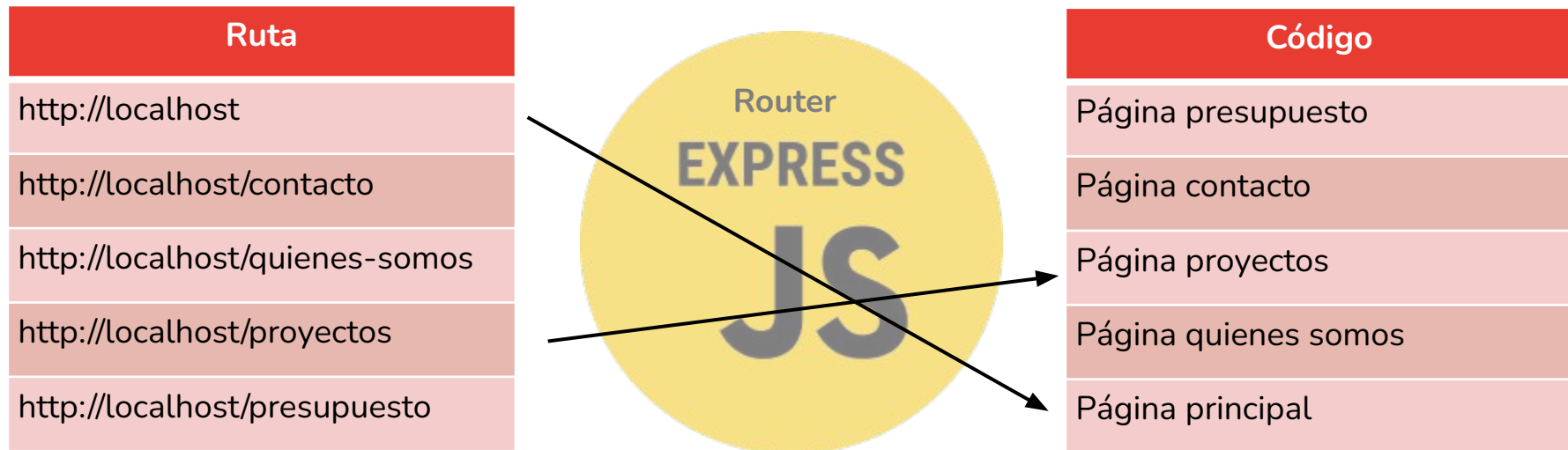
Código
Página presupuesto
Página contacto
Página proyectos
Página quienes somos
Página principal



express - Sistema de enrutado

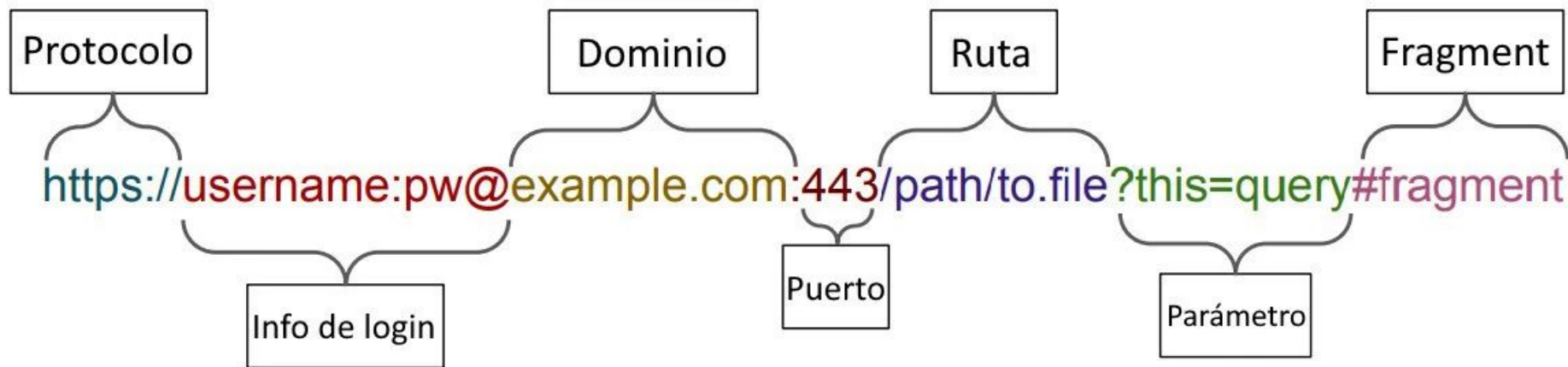
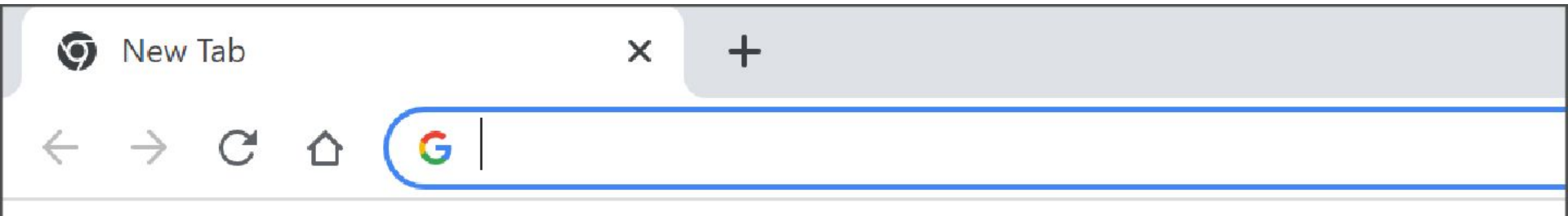
Es un sistema básico de cualquier framework orientado a servicios web ya que cualquier solicitud sobre la información (consultar, modificar, eliminar...) parte de una petición a una url (dónde está el recurso).

El sistema de enrutado, se encarga de ejecutar la parte del código que nosotros especifiquemos cuando se llama a una ruta.



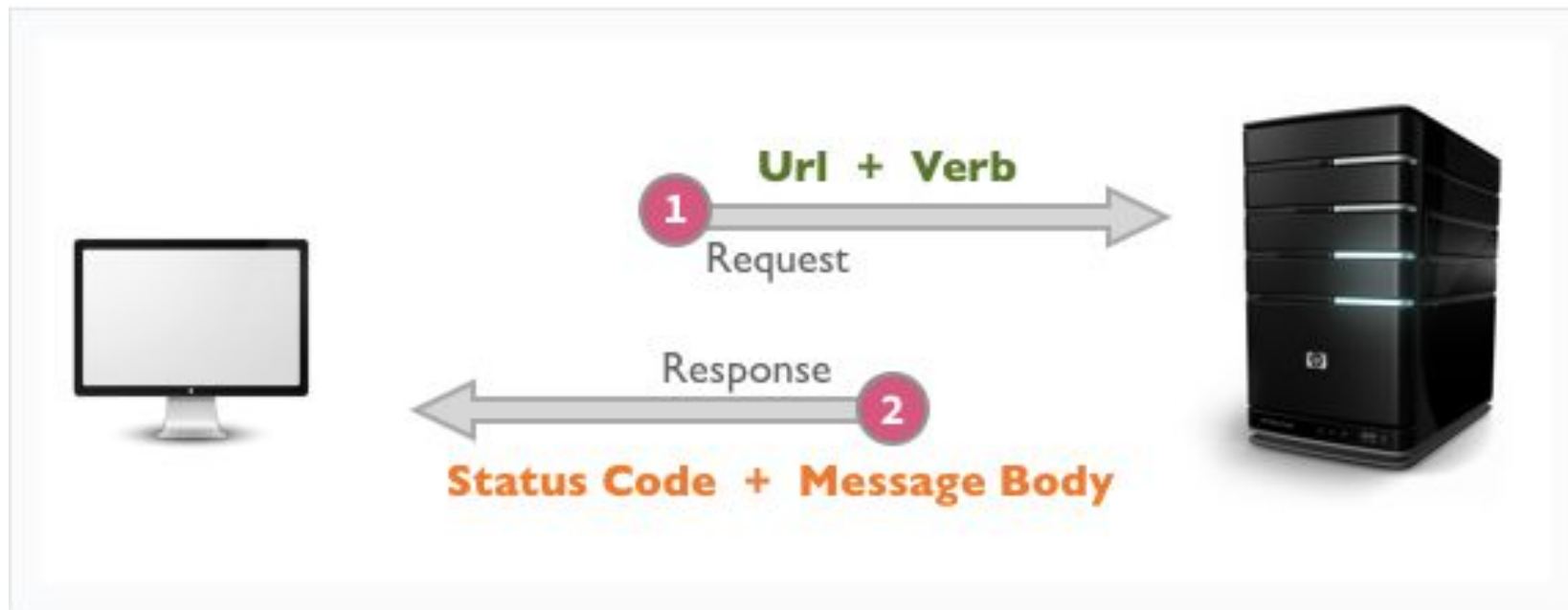


express - Partes de una ruta





express - Estructura mínima petición / respuesta (protocolo HTTP)



express - Creación de rutas

```
//creación de una ruta GET  
app.get('/', (req, res) => {  
  res.send('¡Hola mundo!')  
})
```

```
//creación de una ruta POST  
app.post('/', (req, res) => {  
  res.send('POST enviado a la página principal')  
})
```


express - Parámetros en rutas (I)

```
//parametrización de una ruta
//? - nos permite hacer opcional el parámetro por lo que coincidiría con / y /<lo-que-sea>
app.get('/:nombre?', (req, res) => {
  if (req.params.nombre){
    res.send("Hola " + req.params.nombre)
  }else{
    res.send("Hola anónimo")
  }
})
```

express - Parámetros en rutas (II)

```
//parametrización de una ruta
app.get('/usuario/:apellidos/:nombre', (req, res) => {
  res.send(req.params)
})
```

- Se pueden crear rutas para cualquier método soportado por HTTP (GET, POST, PATCH, OPTIONS, DELETE...)
- Es posible usar expresiones regulares para la creación de rutas

express - Ficheros virtuales

```
//acceso al fichero virtual robots.txt
app.get('/robots.txt', (req, res) => {
  res.writeHead(200, {"Content-Type": "text/html;charset=UTF-8"})
  res.write('User-agent: *\nAllow: /');
  res.end()
})
```

express - Encadenando rutas (chaining)

```
app.route('/usuario')
  .get((req, res) => {
    res.send('Obtiene un usuario aleatorio')
  })
  .post((req, res) => {
    res.send('Crea un usuario')
  })
  .put((req, res) => {
    res.send('Actualiza un usuario')
  })
  .delete((req, res) => {
    res.send('Borra un usuario')
  });
```



express - Request

Algunas propiedades disponibles en el objeto req

Propiedad	Descripción
<code>req.app</code>	Referencia a la aplicación express
<code>req.baseUrl</code>	Cadena con la ruta base de la petición p.e.: /contacto
<code>req.originalUrl</code>	Cadena con la ruta base y parámetros p.e.: /contacto?campaign=x12
<code>req.hostname</code>	Cadena con el hostname
<code>req.ip</code>	Cadena con la dirección ip que ha realizado la petición
<code>req.body</code>	Contenido recibido en el body
<code>req.params</code>	Objeto con los parámetros enviados como parte de la ruta
<code>req.path</code>	Cadena con la parte de la ruta
<code>req.query</code>	Objeto que contiene una key por cada parámetro recibido (no son confiables por lo que hay que validarlos)



express - Response

Algunos métodos disponibles en el objeto res

Método	Descripción
res.download()	Solicita un archivo para descargarlo.
res.end()	Finaliza el proceso de respuesta.
res.json()	Envía una respuesta JSON.
res.redirect()	Redirecciona una solicitud.
res.render()	Renderiza una plantilla con los datos enviados.
res.send()	Envía una respuesta de varios tipos.
res.sendFile()	Envía un archivo como una secuencia de octetos.
res.sendStatus()	Establece el código de estado de la respuesta y envía su representación de serie como el cuerpo de respuesta.

Ejemplo. Calculadora front + back



Descripción

Veamos un ejemplo ya desarrollado de un servicio en backend con express que realiza las operaciones matemáticas ordenadas por una interfaz en el navegador.

A X 1 O S



Uso básico de axios

```
// Instalación standalone  
npm install axios
```

Configuración

```
// src/plugins/axios.js  
import axios from 'axios';  
  
const instance = axios.create({  
  baseURL: 'https://pokeapi.co/api/v2', // Ruta base de todas las llamadas  
  timeout: 10000, // Puedes ajustar el tiempo de espera si es necesario (en milisegundos)  
});  
  
const response = await instance.get('/ruta-del-endpoint');  
  
export default instance;
```

Práctica. Manejo de parámetros

Instrucciones

Práctica. Mi primer web en express

Instrucciones

Descargar las siguientes plantillas.

1.- Crear un servidor NodeJS que gestione las siguientes urls

Página	Ruta	Plantilla
Home	/	index.html
Generic	/generic	generic.html
Elements	/elements	elements.html

2.- Crear un contador de visita.

Notas

1.- No te preocupes por assets externos como imágenes, css, js, fuentes, etc. Están alojados externamente

Bonus track

1.- Crear sección con la ruta **stats** con un nuevo template stats.html generado a partir de index.html. Elimina el contenido (menos cabecera y pie de página) e inserta la información de cada sección donde se pueda ver el nombre y número de visitas.



nodemon
Live reloading



nodemon

El paquete [nodemon](#) nos permite utilizar la funcionalidad conocida como **live reloading** la cual monitoriza los ficheros en nuestro proyecto y lo recarga si detecta cambios sobre alguno de los ficheros.

Ficheros monitorizados según su extensión:

- .js, .mjs (ficheros JavaScript)
- .coffee, .litcoffee (ficheros CoffeeScript)
- .json (ficheros JSON)

YARN - Ejecución de scripts con nodemon

```
//instalando nodemon
```

```
> npm install --save-dev nodemon
```

```
//arrancando nuestro servidor con nodemon
```

```
> nodemon index.js
```

```
//Añadiendo tipos de fichero a la monitorización
```

```
> nodemon --watch . -e css,html index.js
```



Variables de entorno

dotenv



dotenv

El paquete [dotenv](#) permite precargar como variables de entorno elementos de configuración para nuestra aplicación, por ejemplo:

- **Parámetros de ejecución**
- **Cadenas de conexión a bases de datos**
- **Credenciales de acceso a diferentes servicios**

```
//instalar paquete  
> npm install --save-dev dotenv
```

Securiza y parametriza nuestra aplicación. Evitar devolver y/o logar un error o traza con datos sensibles.



dotenv

Para usar adecuadamente las variables de entorno es necesario:

- **Ignorar** los ficheros .env con la configuración mediante .gitignore
- **Crear** un fichero .env.example con la plantilla a seguir e instrucciones para saber cómo cumplimentar cada variable y para qué se utiliza
- **No subir el fichero .env** al servidor en producción. Las variables deben estar ya precargadas en la memoria de la máquina donde correrá nuestra aplicación.

```
# .env.example

# Server port
SERVER_PORT=8080

# Current API Version
API_VERSION=1.0.0
```



dotenv + nodemon

Para precargar las variables de entorno en el fichero .env, hay que realizar un pequeño cambio en el comando que se ejecuta con nodemon para arrancar nuestra aplicación:

```
{  
  ...  
  "scripts": {  
    "dev": "nodemon -r dotenv/config -e html,js,json index.js"  
  }  
}
```

Todas las variables de entorno estarán disponibles en process.env:

```
console.info(process.env.SERVER_PORT)  
console.info(process.env.API_VERSION)
```

Práctica. Configuración servidor node desde dotenv

.ENV

Instrucciones

- 1.- **Crear** un nuevo proyecto en express
- 2.- **Instala el paquete dotenv e intégralo** de la forma más básica posible
- 3.- **El puerto** de la app debe ser configurable desde aquí
- 4.- **El mensaje** a devolver en la ruta / debe ser configurable desde aquí
- 5.- **Cambia la configuración y realiza pruebas**

Notas

- 1.- **Configura** .gitignore adecuadamente
- 2.- **Crea** un .env.template para que otras personas sepan qué variables son necesarias



Node Entornos



Entornos

Los entornos son los diferentes **contextos** en los que se puede ejecutar una aplicación. Entendemos por contextos cada conjunto de parámetros de configuración que modifican el comportamiento de nuestra aplicación.

Los principales entornos de ejecución durante el desarrollo son:

- **development (dev)** Para la fase de desarrollo.
- **testing (test)** Usado durante la ejecución de tests.
- **production (prod)** Usado para un uso real.



Entornos

Node tiene por convención establecer el entorno de ejecución en una variable de entorno llamada **NODE_ENV**.

```
# .env

# enviroment
NODE_ENV=development
NODE_ENV=production
NODE_ENV=testing

# Ejecución app.js con la variable de entorno NODE_ENV a development
> NODE_ENV=development node app.js
```

Dependiendo del valor, podemos iniciar o configurar ciertos módulos de manera diferente, por ejemplo, la fuente de datos a usar, el puerto donde se levantará el servidor o si deben mostrarse errores en pantalla.



express Middlewares



*“Node y por ende express es un framework **basado en middlewares**.*

***Podemos crear** los nuestros **propios o** usar de **terceros** ”*



Middleware - ¿Qué es?

*“Un middleware es un **trozo de código (función)** que se ejecuta **entre** una **petición** que hace el usuario (request) **hasta** que se devuelve la **respuesta** de ésta (response).”*



Middleware - ¿Qué es?

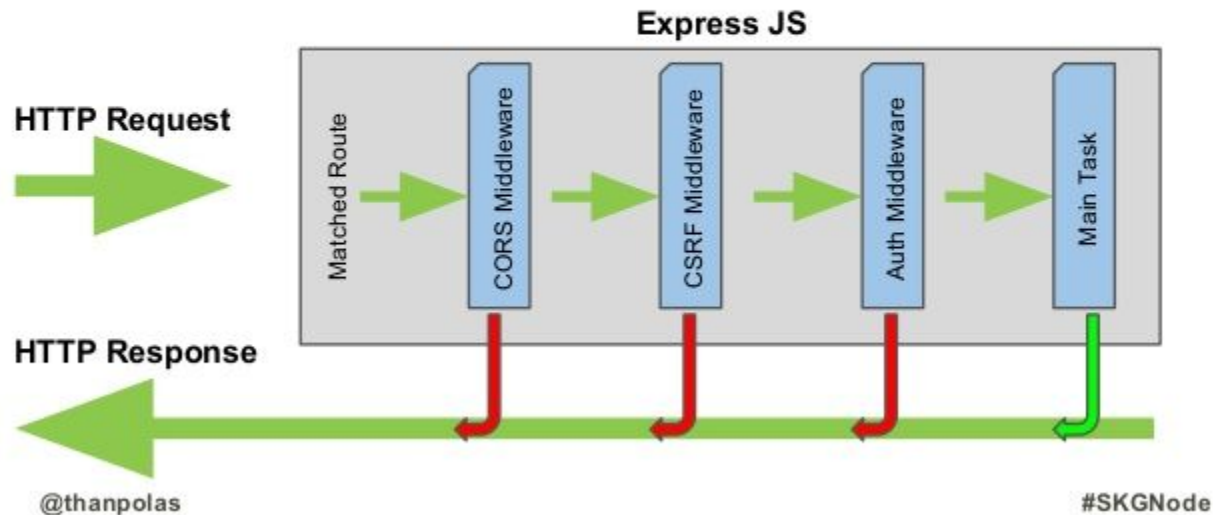
Un middleware es básicamente una función a la que se le inyectan 3 parámetros:

```
function myMiddleware (req, res, next) {  
  console.info("hey from myMiddleware")  
  next()  
}
```

La ejecución de nuestra aplicación puede pasar a través de ella si llamamos al método next o cortar la ejecución si lanzamos una excepción o pasamos un error en el next.



express - Cadena de montaje con middlewares



CSRF - Cross-site request forgery
CORS - Cross-Origin Resource Sharing



express - Potencialidad de los middleware

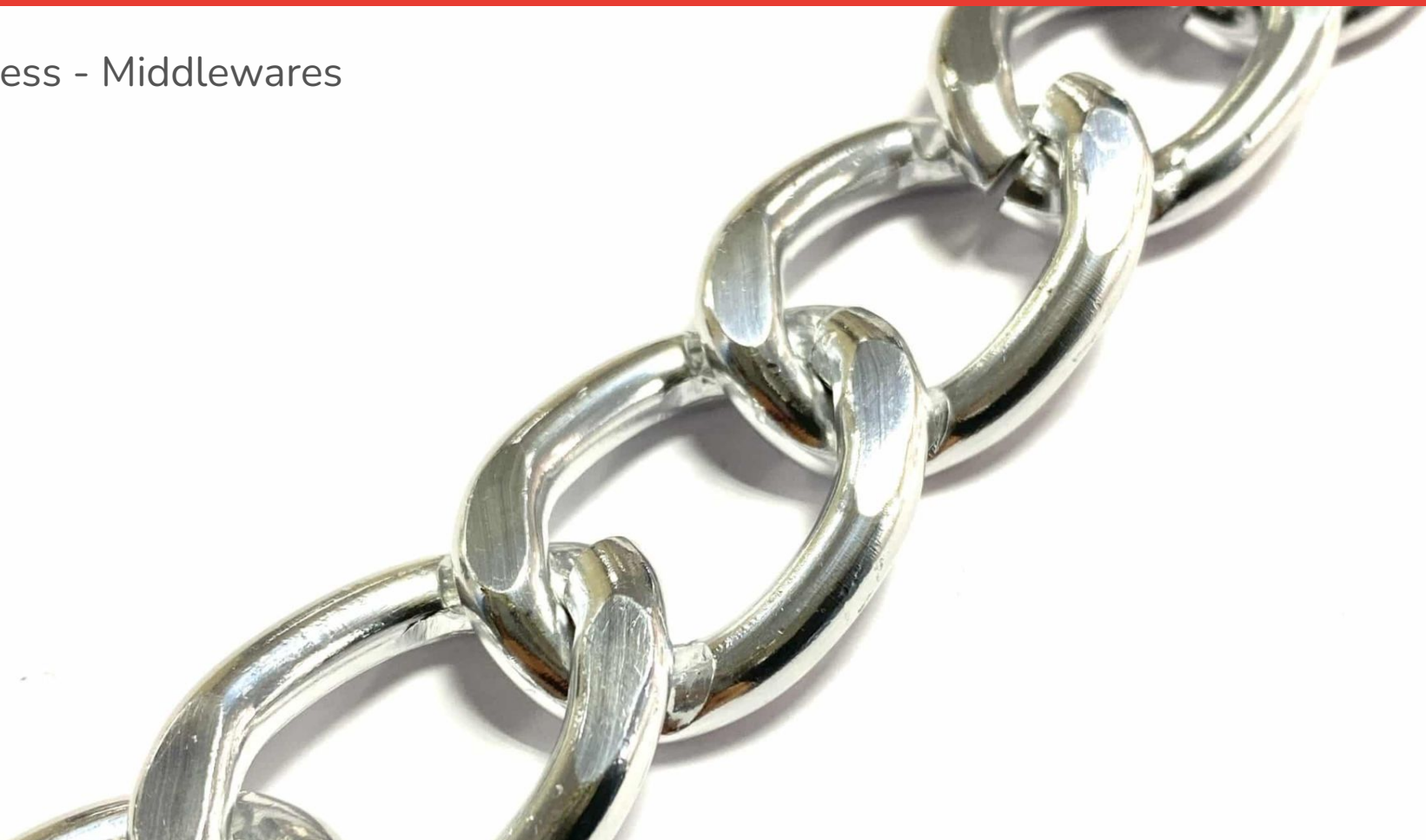
Un Middleware puede:

- Ejecutar cualquier tipo de código
- Modificar la petición del cliente y/o la respuesta
- Finalizar la petición enviando la respuesta al cliente
- invocar al siguiente middleware de la cola

¡Son muy sencillos de usar!



express - Middlewares



express - Cómo enganchar un middleware a express

```
import express from 'express';  
import middleware from 'custom-middleware';  
  
const app = express();  
  
// Enganchamos el middleware y listo para ejecutarse  
app.use(middleware);
```



Método que engancha el middleware a nuestra aplicación

express - Ejemplo de middlewares básicos

```
//Middleware final
app.use((req, res) => {
  res.send("Número de visitas: " + app.locals.counter)
  res.end()
})

//Middleware encadenable (pasa el testigo al que corresponda)
app.use((req, res, next) => {
  app.locals.counter += 1
  next()
})
```

express - Middleware chaining (I)

Referencia a método next



```
app.get('/usuario/:nombre', (req, res, next) => {  
  console.log('La respuesta será enviada a la próxima función...')  
  next()  
}, (req, res) => {  
  res.send('¡Soy la próxima función!')  
})
```


express - Middleware chaining (II)

```
const firstCallBack = (req, res, next) => {  
  console.log('FCB1')  
  next()  
}
```

```
const secondCallBack = (req, res, next) => {  
  console.log('FCB2')  
  next()  
}
```

no hay tercer parámetro



```
const thirdCallBack = (req, res) => {  
  console.log('FCB3')  
}
```

```
app.get('/usuarios/:name', [firstCallBack, secondCallBack, thirdCallBack])
```



express - Middlewares ya incorporados

→ **express.json**

Permite manejar los payloads JSON mandados como parámetros a una ruta (basado en body-parser)

→ **express.urlencoded**

Permite manejar las peticiones url encoded enviadas a una ruta (basado en body-parser)

→ **express.static**

Permite servir ficheros estáticos

express - Configurando el middleware static

```
const options = {
  dotfiles: 'ignore', //ignora ficheros comenzados en .
  etag: false, //añade si true identificador al estático (para cacheado)
  extensions: ['htm', 'html'], //si no se especifica extensión en el fichero busca por la lista
  index: false, //si debe devolver un listado del directorio si no se especifica un fichero
  maxAge: '1d', //tiempo de cacheo
  redirect: false, //redirigir si la ruta no finaliza en /
  setHeaders: function (res, path, stat) {
    res.set('x-timestamp', Date.now())
  }
}

//enganchamos el middleware de forma global con use
app.use(express.static('public', options)) //expone directorio public con opciones
app.use(express.static('public')) //expone directorio public sin opciones
app.use('/statics', express.static('public')) //expone directorio public con prefijo
```



MIDDLEWARES DE TERCEROS

A tener en cuenta



Middleware - Body parser

body-parser

Middleware para Node encargado de transformar (parsear) el body en las peticiones acorde a nuestras necesidades a un objeto.

- No gestiona multipart bodies [Alternativas](#)
- Parsea contenido JSON, Text, URL encoded y RAW

[Página oficial](#)

Middleware - Configurando body-parser en expressJS

```
import express from 'express';
import bodyParser from 'body-parser';

const app = express();

// Habilita el parseo de application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: false }));

// Habilita el parseo de application/json
app.use(bodyParser.json());

// Middleware alternativo a body-parser (incluido en express)
app.use(express.json());
```



Middleware - Morgan

morgan

~~**freeman**~~

Librería para expressJS que funciona como un middleware para la **creación de loggers HTTP** para nuestras aplicaciones.

- Configurable
- No intrusivo
- Loga la request al ser recibida evitando carencias en nuestro log si el servidor se cuelga

[Página oficial](#)



Middleware - Passportjs



Librería especializada en autenticación para Node

- ¡+500 estrategias de autenticación disponibles!
- No intrusivo (es un middleware)
- Simplifica muchísimo la tarea

Hace que las cosas funcionen “like a charm”.

[Página oficial](#)



Middleware - Serve static

serve-static

Librería que funciona como middleware especializado en la gestión de ficheros estáticos

- Altamente configurable
- Sencilla de usar
- Permite configuraciones para diferentes path

[Página oficial](#)



MIDDLEWARES PROPIOS

Creación de middlewares

express - Creación de un middleware propio (contador de visitas)

```
import express from 'express';  
  
const app = express();  
const port = process.env.PORT || 8080;  
app.locals.counter = 0;
```

```
app.use((req, res, next) => {  
  app.locals.counter += 1;  
  next();  
});
```



Nuestro middleware encargado de contar las visitas

```
app.get('*', (req, res) => {  
  res.send("Número de visitas: " + app.locals.counter);  
});  
  
app.listen(port);
```

express - Creación de un middleware propio (contador de visitas) II

```
const express = require('express')
const app = express()
const port = process.env.PORT || 8080
app.locals.counter = 0
```

```
const myMiddleware = (req, res, next) => {
  app.locals.counter += 1
  next()
}
```



Encapsulado en una variable de nuestro middleware encargado de contar las visitas

```
app.use(myMiddleware)
```

```
app.get('*', (req, res) => {
  res.send("Número de visitas: " + app.locals.counter)
})
```

```
app.listen(port)
```

express - Creación de un middleware propio (contador de visitas) II

```
const express = require('express')
const app = express()
const port = process.env.PORT || 8080
app.locals.counter = 0
```

```
let myMiddleware = (req, res, next) => {
  app.locals.counter += 1
  next()
}

app.get('/', myMiddleware, (req, res) => {
  res.send("Número de visitas: " + app.locals.counter)
})
```

```
app.listen(port)
```



Asignación a una ruta concreta



MIDDLEWARES PROPIOS

Middleware configurable

express - Creación de un middleware propio configurable

```
const express = require('express')
const app = express()
const port = process.env.PORT || 8080
app.locals.counter = 0
```

```
let versionMiddleware = (currentAPIVersion) => {
  return (req, res, next) => {
    res.set('Accept-version', currentAPIVersion)
    next()
  }
}
```



Se crea mediante una arrow function (por ejemplo) que devuelve el middleware ya configurado

```
app.get('/', versionMiddleware('v1.1.24'), (req, res) => {
  res.send("Número de visitas: " + app.locals.counter)
})
```



Se llama a la arrow function con los parámetros que acepte

```
app.listen(port)
```

Práctica. Crear un middleware propio

Instrucciones

Crear un middleware que establezca una cabecera personalizada en todas las respuestas con el nombre **Accept-version** que contenga el valor **v1.0**.

Notas

Refactorizar el middleware para que sea **parametrizable** de forma que la versión venga de un parámetro y que el valor de éste sea **inyectado desde una variable de entorno**.



express

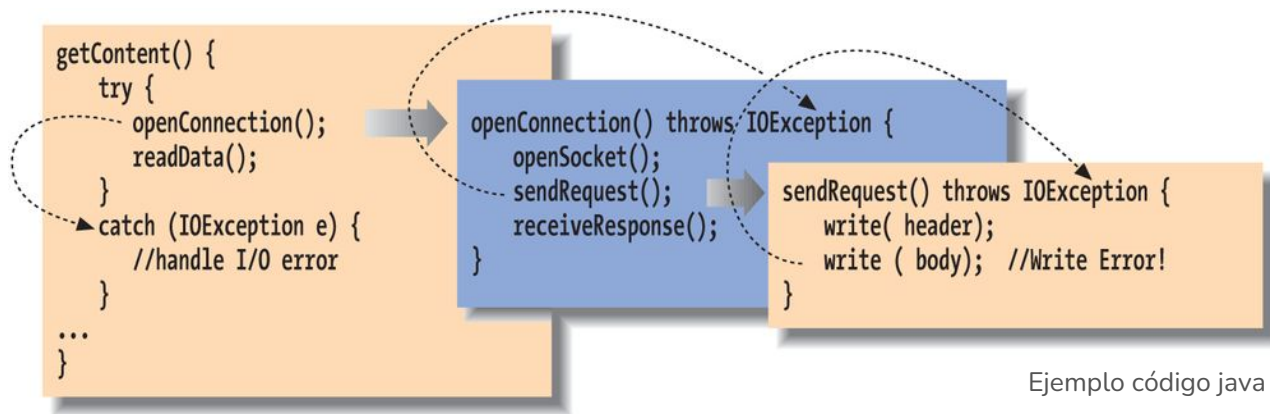
Manejo de errores

The background of the slide is a close-up, high-magnification photograph of water. It is filled with numerous bubbles of various sizes, from tiny specks to larger, more defined spheres. The water has a deep blue-green hue, and the lighting creates bright highlights on the surfaces of the bubbles, giving them a three-dimensional appearance. The overall effect is one of dynamic movement and texture.

Error bubbling



Error bubbling



Al producirse un error es lanzado hacia arriba (nivel de código superior) desde que son lanzados en lo que se conoce como “error bubbling” ya que se asemeja a el ascenso de una burbuja a la superficie.

Es muy recomendable que los errores sean manejados



express - Manejo de errores

➔ Express debe recibir los errores que se producen en los middlewares de manejo de url's

```
app.get('/', (req, res) => {  
  throw new Error('error') // expressJS capturará este error  
})
```

express viene ya con un manejador de errores por defecto (incluido al final de todos los middlewares).

Si no hemos creado un manejador personalizado, atrapa los errores y muestra con una traza del error.

Si el entorno `NODE_ENV` es **production** no incluye la traza del error para no revelar la estructura interna del proyecto a posibles atacantes.



express - Manejo de errores

→ Pasando errores en funciones asíncronas

```
app.get('/', (req, res, next) => {  
  fs.readFile('/fichero-no-existe.txt', (err, data) => {  
    if (err) {  
      next(err) // pasa el error a express  
    } else {  
      res.send(data)  
    }  
  })  
})
```



express - Manejo de errores

➔ Pasando errores en bloques try...catch

```
app.get('/', (req, res, next) => {  
  setTimeout(() => {  
    try {  
      throw new Error('error')  
    } catch (err) {  
      next(err)  
    }  
  }, 1000)  
})
```



express - Manejo de errores

→ Pasando errores en promesas

```
app.get('/', (req, res, next) => {  
  Promise.resolve().then(() => {  
    throw new Error('BROKEN')  
  }).catch(next) // el error será enviado a expressJS  
})
```



express - Manejo de errores

- Si se llama a next() con un error, el manejador por defecto de express cerrará la conexión y la marcará como errónea
- Es posible crear manejadores personalizados de errores, más información [aquí](#).

```
app.use((err, req, res, next) => {  
  // express recomienda delegar en ellos la gestión cuando las cabeceras ya se enviaron  
  if (res.headersSent) {  
    return next(err);  
  }  
  
  console.error(err.stack)  
  res.status(500).send(';algo está roto!')  
  //en el send podemos cargar la plantilla de nuestra página de error 500, 404...  
})
```




express - Página no encontrada

- Las respuestas de página no encontrada no son el resultado de un error, por lo que el middleware de manejador de errores no las capturará.
- Lo único que debe hacer es añadir una función de middleware al final de todas la rutas.

```
app.get('/', (req, res) => res.send('página principal'))
app.get('/contacto', (req, res) => res.send('página contacto'))
app.get('/nosotros', (req, res) => res.send('página nosotros'))
//manejador de página no encontrada ya que si se hubiera ejecutado algún middleware
enganchado
//a una ruta (los cuales no llaman a next) la ejecución no llegaría a este middleware
app.use((req, res, next) => {
  res.status(404).send('página no encontrada');
})
```

Práctica. Manejo de errores

Instrucciones

Teniendo como referencia el ejercicio **Mi primera web en express**, hacer que al solicitar una página que no exista, devuelva un mensaje de error 404.

